Optimal Control, Differential Games, Continuous-Impulsive Control, and Games on Networks

A Self-Contained English Tutorial Note with Degree-Level and Node-Level Examples

Lu-Xing Yang

Created from my research experience in optimal control, differential games, impulse and continuous–impulsive hybrid control, and cyber/network-security applications

Informed by publications in IEEE TIFS, TDSC, TSMC, TNSE, TCSS, and related journals

Tutorial Python files: `simple_degree_k_control.py` and `network_control_examples.py`

Purpose. This note starts from controlled dynamical systems and develops optimal control, differential games, and impulse and continuous–impulsive hybrid control in one coherent framework. It is written for propagation, defense, and intervention problems on networks. The mathematical sections introduce the state equation, payoff, Hamiltonian, PMP, HJB/HJI viewpoint, Nash equilibrium conditions, impulse jumps, and continuous-plus-impulse strategies. The computational sections give a simple degree-k example first and then a main tutorial script for degree-level games, node-level adjacency-matrix models, and continuous-impulse simulations. The degree-level workflow always computes N_k, $P(k)$, and $\langle k\rangle$ from the input network before solving models indexed by degree class k.

July 7, 2026

Contents

1 How to Read This Handout **3**

2 Dynamic Systems and Network Propagation Models **4**
2.1 State, Control, and Objective . . . 4
2.2 Degree-Level Models . . . 4
2.3 Node-Level Models . . . 6
2.4 From a Real Network Dataset to the Adjacency Matrix . . . 6

3 Optimal Control: Single Decision Maker **7**
3.1 General Model . . . 8
3.2 Hamiltonian and Pontryagin Maximum Principle . . . 8
3.3 Forward-Backward Sweep . . . 8
3.4 Important Practical Cautions . . . 9

4 Differential Games: Multiple Strategic Decision Makers **9**
4.1 General Two-Player Nonzero-Sum Open-Loop Nash Game . . . 9
4.2 Classification of Differential Games . . . 10
4.3 PMP for an Open-Loop Nash Differential Game . . . 10
4.4 Game-Theoretic Interpretation . . . 11

5 Flow, Impulse, and Continuous–Impulsive Models **11**
5.1 Continuous-Time Control . . . 11
5.2 Impulse Control . . . 11
5.3 Continuous–Impulsive Hybrid Control . . . 12
5.4 Continuous–Impulsive Hybrid Differential Games . . . 12

6 Theoretical Core: PMP, HJB, Impulse Jumps, and Optimality Conditions **13**
6.1 Necessary Conditions versus Sufficient Conditions . . . 13
6.2 Continuous-Time PMP: State, Adjoint, Stationarity, and Transversality . . . 14
6.3 Sufficiency for Continuous Optimal Control . . . 15
6.4 HJB and the Verification Theorem . . . 15
6.5 HJB/HJI Equations for Games . . . 16
6.6 Impulse HJB and Quasi-Variational Inequalities . . . 16
6.7 PMP with Impulsive State Jumps . . . 17
6.8 Continuous–Impulsive Hybrid PMP and Nested Optimization . . . 18
6.9 Impulse and Continuous–Impulsive Hybrid Differential Games . . . 18
6.10 A Short Theoretical Checklist . . . 19

7 Degree-Level and Node-Level Examples **19**
7.1 Degree-Level Optimal Control: Degree-k Controlled SIS Spreading . . . 19
7.2 Node-Level Optimal Control: Targeted Patching . . . 20
7.3 Degree-Level Attacker-Defender Differential Game . . . 20
7.4 Node-Level Attacker-Defender Differential Game . . . 21
7.5 Continuous–Impulsive Node-Level Control: Continuous Patching plus Emergency Cleanup . . . 21

8 Numerical Solution Methods and Practical Cautions **21**

8.1 Forward-Backward Sweep for Games . . . 22
8.2 Direct Optimization and Collocation . . . 22
8.3 Why HJB is Difficult in Network Models . . . 22
8.4 Model Checking and Baseline Comparison . . . 22

9 Python Tutorial Code: Simple First, Then Research-Code-Style Examples 23
9.1 Tutorial Files . . . 23
9.2 Run the Simple Degree-k Example . . . 23
9.3 Run the Main Examples . . . 24
9.4 How the Reference Repositories Fit In . . . 25
9.5 Representative Outputs . . . 26

10 Glossary and References 29
10.1 Glossary . . . 29
10.2 References and Source Notes . . . 29

A Tutorial Code Files 30

1 How to Read This Handout

This handout follows a layered structure. First, it defines a controlled dynamical system. Second, it formulates an optimal control problem for one decision maker. Third, it extends the same logic to two or more strategic decision makers, producing a differential game. Fourth, it adds discrete state jumps and event decisions, producing impulse and continuous–impulsive hybrid control/game models. Finally, it connects the mathematics to degree-level and node-level propagation models on networks and to runnable Python implementation.

Core questions

- **Optimal control**: what should one controller do over time to maximize benefit or minimize cost?
- **Differential game**: what should each player do when the system state is jointly affected by all players and their objectives may conflict?
- **Impulse control/game**: when and how should a player apply instantaneous actions that cause state jumps?
- **Continuous–impulsive hybrid control/game**: how should continuous-time controls and discrete impulse decisions be combined?
- **Degree-level versus node-level networks**: should we group nodes by type/degree, or track every node and use the full adjacency matrix?

Table 1: A layered view of the guide.

Layer	**Mathematical object**	**Typical question**	**Network example**
Continuous dynamics	$\dot{x} = f(x, u, t)$	How does the state evolve under continuous effort?	Patch rate, recovery effort, attack rate over time
Optimal control	$\max_u J(u)$ subject to dynamics	What is the best control for one decision maker?	Defender allocates patching to reduce malware
Differential game	$\max_{u_i} J_i(u_i, u_{-i})$ for each player	What is a Nash or Stackelberg equilibrium?	Attacker increases spread; defender suppresses spread
Impulse control	$x(t_l^+) = x(t_l^-) + g(x(t_l^-), v_l)$	When should discrete interventions be applied?	Emergency cleanup, forced password reset, public correction campaign
Continuous–impulsive hybrid control/game	Continuous dynamics plus impulse resets	How should continuous flow controls and impulse decisions be combined?	Routine monitoring plus occasional large-scale response

Notation conventions used throughout the note

The same mathematical object can be written differently in different communities. This note uses the following convention unless explicitly stated otherwise.

Symbol	Meaning
T	finite time horizon; the control/game runs on $[0, T]$.
$x(t) \in \mathbb{R}^n$	state vector. In node-level models, $x_i(t)$ is the infection, compromise, belief, or risk level at node i.
$u(t), v(t)$	controls. In games, different symbols usually belong to different players.
$A = [a_{ij}]$	adjacency matrix. In this note, $(Ax)_i = \sum_j a_{ij} x_j$ is the pressure on node i from its neighbors.
F, L	running payoff or running loss. We use maximization with F and minimization with L.
H	Hamiltonian. For games there is one Hamiltonian per player.
$\lambda(t)$	adjoint or costate. It is solved backward in time and acts as a shadow value of the state.
t_l, v_l	impulse time and impulse magnitude at event l.

2 Dynamic Systems and Network Propagation Models

A dynamic system describes how a state changes over time. In cybersecurity, information diffusion, epidemiology, and social network analysis, the state may measure infection, compromise, awareness, rumor belief, or defense status. A control modifies the state equation. A game appears when two or more decision makers control the same system with different objectives.

2.1 State, Control, and Objective

Let the time horizon be $[0, T]$. Let $x(t) \in \mathbb{R}^n$ be the state vector and $u(t) \in U$ be the control. A deterministic continuous-time controlled system is written as

$$\dot{x}(t) = f(x(t), u(t), t), \qquad x(0) = x_0, \qquad 0 \le t \le T. \tag{1}$$

The controller usually optimizes a cumulative payoff or cost. A maximization form is

$$J(u) = \Phi(x(T)) + \int_0^T F(x(t), u(t), t)\, \mathrm{d}t. \tag{2}$$

Here F is the running payoff and Φ is the terminal payoff. In a minimization formulation, F is often a running cost such as infection loss plus control cost.

2.2 Degree-Level Models

Degree-level state means degree-indexed state

A degree-level model should not be implemented as a single scalar compartmental SIR/SIS model unless the network is being ignored. In a network-aware degree-level model, the first step is to compute the degree classes from the input graph. If N_k is the number of nodes with degree k and n is the total number of nodes, then

$$P(k) = \frac{N_k}{n}, \qquad \bar{k} = \frac{1}{n}\sum_{i=1}^{n} k_i = \sum_k kP(k). \tag{3}$$

The state is then arranged as

$$\begin{aligned} x(t) &= \big(x_k(t)\big)_{k\in\mathcal{K}}, \qquad \mathcal{K} = \{k : N_k > 0\}, \\ x_k(t) &= \text{infected/compromised fraction among degree-}k\text{ nodes.} \end{aligned} \tag{4}$$

A common heterogeneous mean-field infection pressure is

$$\Theta(t) = \frac{\sum_{k\in\mathcal{K}} kP(k)x_k(t)}{\langle k\rangle}, \tag{5}$$

where $P(k)$ is the degree distribution and $\langle k\rangle = \bar{k}$ is the average degree. A controlled degree-level SIS-style model is

$$\dot{x}_k(t) = \beta k(1 - x_k(t))\Theta(t) - \big(\delta + u_k(t)\big)x_k(t), \qquad k \in \mathcal{K}. \tag{6}$$

The implementation therefore stores one state and one control trajectory per observed degree value. For example, if the input network has degree classes $\mathcal{K} = \{2, 3, 4, 7\}$, then the model state is (x_2, x_3, x_4, x_7), not (S, I, R).

The code also supports class-specific physical and economic parameters. With susceptibility s_k, infectivity ι_k, recovery δ_k, and row-stochastic mixing $M_{k\ell}$, the heterogeneous pressure is

$$\lambda_k(t) = s_k k \sum_{\ell\in\mathcal{K}} M_{k\ell}\iota_\ell x_\ell(t), \qquad \dot{x}_k(t) = (1 - x_k(t))\lambda_k(t) - (\delta_k + u_k(t))x_k(t).$$

Scalar inputs are explicitly broadcast by `cybercontrol.heterogeneity`, so the homogeneous model remains a special case.

In the tutorial code, `compute_degree_distribution` returns the observed degree values, counts N_k, probabilities $P(k)$, and average degree $\langle k\rangle$. For directed networks, one must also decide what "degree" means. The code exposes a `degree_mode` option: row degree counts incoming influencers under the convention $\dot{x} = f(Ax)$, column degree counts outgoing influence, total degree adds both, and undirected degree ignores orientation.

Degree-level models are compact and interpretable. They are useful when the network is large or only statistical network information is available. Their limitation is that they lose fine node-level structure: which node is connected to which node, which node is high value, and where attacks or defenses should be targeted.

2.3 Node-Level Models

A node-level model tracks each node. Let $A = [a_{ij}]$ be the adjacency matrix. Let $x_i(t)$ be the probability or intensity that node i is infected or compromised. A standard controlled node-level model is

$$\dot{x}_i(t) = \beta(1 - x_i(t)) \sum_{j=1}^{n} a_{ij} x_j(t) - (\delta + u_i(t)) x_i(t), \qquad i = 1, \ldots, n. \tag{7}$$

Here β is the propagation rate, δ is the natural recovery/removal rate, and $u_i(t)$ is node-specific defense effort. This model explicitly uses network connectivity and supports targeted control. The cost is computational: the state dimension grows with n, and the adjoint or HJB equation becomes harder as the network grows.

2.4 From a Real Network Dataset to the Adjacency Matrix

Real network datasets are often not distributed as adjacency matrices. They are usually stored as an *edge list*: each row records a connection such as

`source target` or `source,target,weight`.

To use such a dataset in this tutorial, the code first parses the full input network and computes the empirical degree distribution N_k, $P(k)$, and $\langle k \rangle$. It then optionally converts a smaller high-degree induced subgraph into a dense adjacency matrix A for node-level PMP/game examples. Thus, degree-level models can use the full-network degree distribution even when node-level examples use a reduced adjacency matrix for computational clarity. The conversion is conceptually simple but important, because mistakes in node indexing, direction, self-loops, or normalization can change the dynamics substantially.

Edge list to adjacency matrix: practical workflow

1. **Read edges.** Parse each line as (source, target) or (source, target, weight).
2. **Relabel nodes.** Map arbitrary node labels such as account IDs or IP addresses to integer indices $0, \ldots, n-1$.
3. **Choose directed or undirected.** For an undirected contact graph, set both a_{ij} and a_{ji}. For a directed influence graph, keep the edge direction.
4. **Remove self-loops and invalid weights.** Most SIS/SIR-style propagation examples assume $a_{ii} = 0$ and nonnegative weights.
5. **Normalize if needed.** Divide by maximum degree, row sums, or spectral radius to keep propagation probabilities numerically stable.
6. **Reduce size if needed for node-level computation.** Full node-level PMP uses dense adjoint/Jacobian calculations. For large datasets, first keep a subgraph such as the top-degree 30–200 nodes, a community, or a sampled ego-network. The degree-level distribution $P(k)$ can still be computed from the full input network before this reduction.

The tutorial code implements this workflow in the single-file functions `load_network`, `graph_to_model_matrix`, and `degree_distribution_from_graph`. The convention used by the dynamics is

$$\text{infection pressure on node } i = (Ax)_i = \sum_{j=1}^{n} a_{ij} x_j. \tag{8}$$

Thus, for a directed edge "source influences target", the code places the edge weight in the matrix row for the target and the matrix column for the source. For an undirected graph, it symmetrizes the matrix.

```python
from network_control_examples import (
    build_parser,
    load_network,
    degree_distribution_from_graph,
    solve_degree_control,
    solve_node_control,
)

args = build_parser().parse_args([
    "--edge-list", "data/edges.txt",
    "--max-nodes", "30",        # only reduces the node-level matrix
    "--degree-mode", "undirected",
])

network = load_network(args)
D = degree_distribution_from_graph(network.full_graph, mode=args.degree_mode)
A = network.A

# Degree-level workflow: columns are observed degree classes k.
degree_result = solve_degree_control(D)

# Node-level workflow: use the dense simulation adjacency matrix A.
node_result = solve_node_control(A)
```

Listing 1: Minimal example: load an edge-list dataset, compute the degree distribution, and run degree-level and node-level models.

Scaling warning

A real network can contain thousands or millions of nodes. The node-level PMP examples in this note intentionally use dense matrices so that the mathematics is transparent. For large datasets, use a subgraph, degree-level aggregation, sparse linear algebra, direct simulation with heuristic controls, or other approximate methods. The full HJB equation is even more expensive because its value function lives on the whole state space.

Table 2: Degree-level versus node-level models.

Aspect	Degree-level model	Node-level model
State dimension	Number of degree classes or compartments	Number of nodes times number of compartments
Network information	Degree distribution or mixing pattern	Full adjacency matrix or contact graph
Best use case	Large networks, aggregate analysis, early theory	Small/medium networks, targeted control, simulation
Main weakness	Less precise for targeted interventions	High dimensionality and possible data requirements

3 Optimal Control: Single Decision Maker

Optimal control is the single-player foundation. It asks for a time-dependent control strategy $u^*(t)$ that optimizes an objective while satisfying the system dynamics. This is the natural

starting point for differential games.

3.1 General Model

$$\max_{u(\cdot)\in U} \quad J(u) = \int_0^T F(x(t), u(t), t)\,\mathrm{d}t, \tag{9}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t), u(t), t), \qquad x(0) = x_0. \tag{10}$$

In a minimization problem, the same structure applies after changing the sign of the objective. For example, minimizing infection and control cost is equivalent to maximizing the negative of that cost.

3.2 Hamiltonian and Pontryagin Maximum Principle

The Hamiltonian combines the immediate payoff and the effect of the control on future state evolution. For a maximization problem,

$$H(x, u, \lambda, t) = F(x, u, t) + \lambda^{\mathsf{T}} f(x, u, t). \tag{11}$$

The adjoint variable $\lambda(t)$ is a shadow value of the state: it measures how a small change in the state affects future payoff. Under standard regularity assumptions, a candidate optimum must satisfy the following necessary conditions:

$$\dot{x}^*(t) = f(x^*(t), u^*(t), t), \qquad x^*(0) = x_0, \tag{12}$$

$$\dot{\lambda}^*(t) = -\frac{\partial H}{\partial x}(x^*(t), u^*(t), \lambda^*(t), t), \qquad \lambda^*(T) = 0, \tag{13}$$

$$u^*(t) \in \arg\max_{u\in U} H(x^*(t), u, \lambda^*(t), t). \tag{14}$$

These equations form a two-point boundary value problem: the state has an initial condition, while the adjoint has a terminal condition. This is why forward-backward sweep is commonly used.

What PMP does and does not prove

PMP gives necessary conditions under regularity assumptions. It does not automatically prove that a computed control is globally optimal. In practice, one should compare the computed policy with baselines, test sensitivity, and report convergence residuals.

3.3 Forward-Backward Sweep

Forward-backward sweep is an iterative numerical method for solving PMP optimality systems. The idea is simple but powerful: guess the control, simulate the state forward, simulate the adjoint backward, update the control from the Hamiltonian condition, and repeat.

Generic forward-backward sweep

1. Initialize $u^{(0)}(t)$, usually zero, constant, or a simple heuristic.
2. Forward step: solve $\dot{x} = f(x, u^{(r)}, t)$ from $t = 0$ to T.
3. Backward step: solve $\dot{\lambda} = -H_x$ from $t = T$ to 0 using the state trajectory.
4. Control update: compute $u_{\text{new}}(t) = \arg\max H$ or $\arg\min H$ depending on the formulation.
5. Relaxation: set $u^{(r+1)} = \theta u_{\text{new}} + (1 - \theta)u^{(r)}$ to improve stability.
6. Stop when the control and state changes are below a tolerance, or after a maximum number of iterations.

```
for iteration in range(max_iter):
    x = forward_simulate(control)
    lambda_adj = backward_simulate(x, control)
    candidate_control = pointwise_hamiltonian_optimizer(x, lambda_adj)
    control = theta * candidate_control + (1 - theta) * control
    if convergence_is_satisfactory:
        break
```

Listing 2: Forward-backward sweep pseudocode.

3.4 Important Practical Cautions

- Forward-backward sweep may fail to converge if the model is stiff, the horizon is long, the control bounds are sharp, or the initial guess is poor.
- A computed control should be compared with no control, constant control, greedy control, random timing, or heuristic resource allocation.
- For high-dimensional node-level systems, HJB is usually infeasible because the value function depends on every state dimension.

4 Differential Games: Multiple Strategic Decision Makers

A differential game is an optimal control problem with multiple players. The same state is affected by multiple controls, and each player has its own payoff. In cybersecurity, the players may be an attacker and a defender. In opinion or propaganda dynamics, they may be competing campaigns. In malware and patching, one player may inject infection effort while the other allocates repair or backup resources.

4.1 General Two-Player Nonzero-Sum Open-Loop Nash Game

Let player 1 choose $u(t)$ and player 2 choose $v(t)$. The system is

$$\dot{x}(t) = f(x(t), u(t), v(t), t), \qquad x(0) = x_0. \tag{15}$$

Each player has a payoff:

$$J_1(u, v) = \int_0^T F_1(x(t), u(t), v(t), t)\, \mathrm{d}t, \tag{16}$$

$$J_2(u, v) = \int_0^T F_2(x(t), u(t), v(t), t)\, \mathrm{d}t. \tag{17}$$

An open-loop Nash equilibrium (u^*, v^*) satisfies

$$J_1(u^*, v^*) \geq J_1(u, v^*) \qquad \text{for all admissible } u, \tag{18}$$

$$J_2(u^*, v^*) \geq J_2(u^*, v) \qquad \text{for all admissible } v. \tag{19}$$

Open-loop means each control is a function of time, usually computed before the game starts. Feedback strategies depend on the current state $x(t)$, but solving feedback differential games often requires HJB or Hamilton-Jacobi-Isaacs equations, which are much harder in high dimensions.

4.2 Classification of Differential Games

Table 3: Common classifications of differential games.

Dimension	**Main types**	**Meaning for applications**
Payoff relation	Zero-sum; nonzero-sum	Zero-sum is pure conflict; nonzero-sum allows asymmetric or partially aligned objectives.
State evolution	Deterministic; stochastic	Deterministic models are easier; stochastic models handle random disturbances and uncertainty.
Decision order	Nash; Stackelberg	Nash players move simultaneously; Stackelberg has leader-follower structure.
Information structure	Open-loop; feedback; sampled-data	Open-loop uses time plans; feedback observes state; sampled-data updates at discrete observation times.
Action type	Continuous-time; impulse; continuous–impulsive hybrid	Continuous-time control changes derivatives; impulse control causes jumps; continuous–impulsive hybrid control contains both.

4.3 PMP for an Open-Loop Nash Differential Game

For each player i, define a player-specific Hamiltonian:

$$H_i(x, u, v, \lambda_i, t) = F_i(x, u, v, t) + \lambda_i^{\mathsf{T}} f(x, u, v, t), \qquad i = 1, 2. \tag{20}$$

A candidate open-loop Nash equilibrium must satisfy the state equation, one adjoint system for each player, and one pointwise best-response condition for each player:

$$\dot{\lambda}_i(t) = -\frac{\partial H_i}{\partial x}(x^*(t), u^*(t), v^*(t), \lambda_i(t), t), \qquad \lambda_i(T) = 0, \quad i = 1, 2, \tag{21}$$

$$u^*(t) \in \arg\max_u H_1(x^*(t), u, v^*(t), \lambda_1(t), t), \tag{22}$$

$$v^*(t) \in \arg\max_v H_2(x^*(t), u^*(t), v, \lambda_2(t), t). \tag{23}$$

The structure is almost identical to optimal control, except that each player has its own adjoint variable and its own Hamiltonian. Numerically, forward-backward sweep updates the state, both adjoints, and both controls iteratively.

4.4 Game-Theoretic Interpretation

- The Nash condition is a mutual best-response condition. No player can improve its own payoff by unilateral deviation, given the other player's strategy.
- For nonzero-sum games, the equilibrium may not be socially optimal. It is strategically stable, not necessarily globally efficient.
- For attacker-defender models, the defender equilibrium effort is often high when infection is high and when the defender adjoint indicates a high future value of reducing infection.
- For propagation games on networks, node centrality and local infection pressure strongly affect the structure of equilibrium controls.

5 Flow, Impulse, and Continuous–Impulsive Models

The key distinction is how an action affects the state. A continuous-time control changes the derivative $\dot{x}$ over an interval of time. An impulse action causes an instantaneous state jump at selected time points. A continuous–impulsive hybrid policy uses both.

5.1 Continuous-Time Control

A continuous-time control $u(t)$ is applied over time and changes the vector field f. Examples include continuous patching intensity, monitoring rate, misinformation refutation effort, or resource allocation to recovery.

$$\dot{x}(t) = f(x(t), u(t), t), \qquad 0 \le t \le T. \tag{24}$$

5.2 Impulse Control

An impulse control has discrete intervention times $t_1, \ldots, t_N$ and impulse magnitudes $v_1, \ldots, v_N$. Between impulses, the system follows a continuous differential equation. At an impulse time, the state jumps:

$$\dot{x}(t) = f(x(t), u(t), t), \qquad t \notin \{t_1, \ldots, t_N\}, \tag{25}$$

$$x(t_l^+) = x(t_l^-) + g(x(t_l^-), v_l, t_l), \qquad l = 1, \ldots, N. \tag{26}$$

A typical impulse objective is

$$J = \int_0^T F(x(t), u(t), t)\,\mathrm{d}t + \sum_{l=1}^{N} G(x(t_l^-), v_l, t_l). \tag{27}$$

In minimization form, G can be an intervention cost. In maximization form, G can be the immediate benefit minus impulse cost.

Continuous versus impulse action

A continuous-time control signal is a function over a time interval and modifies the rate of change. An impulse action is an event action and modifies the state itself. In network security, continuous defense may be monitoring or patching; an impulse may be forced password reset, emergency cleanup, public warning, or manual shutdown of compromised accounts.

5.3 Continuous–Impulsive Hybrid Control

A continuous–impulsive hybrid control combines continuous-time flow controls $u(t)$ and impulse decisions (N, t_l, v_l). In compact form,

$$\dot{x}(t) = f(x(t), u(t), t), \qquad t \notin \{t_1, \ldots, t_N\}, \tag{28}$$

$$x(t_l^+) = x(t_l^-) + g(x(t_l^-), v_l, t_l), \qquad l = 1, \ldots, N. \tag{29}$$

The continuous–impulsive optimization problem is hard because it mixes continuous function optimization and discrete event optimization:

- choose the number of impulses N;
- choose the impulse times $t_1 < \cdots < t_N$;
- choose impulse magnitudes or target sets v_l;
- choose continuous controls $u(t)$ between the impulses.

This naturally leads to nested optimization. The inner layer solves continuous control for fixed impulse schedule. The middle layer optimizes impulse times. The outer layer optimizes the number of impulses. In network propagation models, the exact full solution is often computationally prohibitive. Practical studies may fix impulse times, use heuristics, restrict the number of impulses, or compare several candidate schedules rather than solving the full impulse PMP exactly.

5.4 Continuous–Impulsive Hybrid Differential Games

A continuous–impulsive hybrid differential game appears when at least one player uses continuous-time controls and at least one player can trigger impulses. For example, the defender may continuously patch nodes and occasionally perform emergency cleanup, while the attacker continuously injects effort or occasionally launches bursts. The Nash conditions remain best-response conditions, but the impulse player now has to optimize event count, event timing, targets, and magnitudes.

Table 4: Continuous-time flow control, impulse/reset control, and continuous–impulsive combinations.

Action type	**Effect on state**	**Decision variables**	**Typical method**
Continuous	Changes $\dot{x}$ over time	$u(t)$	PMP + forward-backward sweep; direct collocation; approximate dynamic programming
Impulse	Instantaneous jump $x(t^+) \neq x(t^-)$	N, t_l, v_l	Impulse PMP; direct search; dynamic programming for small problems
Continuous–impulsive hybrid	Both derivative changes and jumps	$u(t)$, N, t_l, v_l	Nested optimization; heuristics; mixed-integer/control methods

6 Theoretical Core: PMP, HJB, Impulse Jumps, and Optimality Conditions

This section adds a more theoretical layer behind the models used in the examples. The main message is that PMP, impulse PMP, and HJB are not competing ideas. They answer different versions of the same question. PMP is usually the most convenient route to *open-loop* controls and open-loop Nash equilibria. HJB is the dynamic-programming route to *feedback* controls. Impulse and continuous–impulsive hybrid models add jump conditions and event-time conditions to the continuous PMP system.

6.1 Necessary Conditions versus Sufficient Conditions

An optimality condition is **necessary** if every true optimum must satisfy it, but a point satisfying it may still fail to be optimal. An optimality condition is **sufficient** if satisfying it guarantees optimality, usually under stronger convexity or concavity assumptions. This distinction is essential when interpreting PMP-based numerical solutions.

How to read PMP results in research papers

For nonlinear propagation models, PMP usually provides a candidate control or candidate equilibrium, not a global proof. A convincing study should report the derived optimality system, describe numerical convergence, and compare the obtained strategy against strong baselines. In differential games, it is also useful to test whether unilateral deviations improve a player's payoff.

Table 5: Necessary and sufficient conditions in control and game models.

Setting	**Typical necessary condition**	**Typical sufficient/verification condition**
Continuous optimal control	State equation, adjoint equation, Hamiltonian stationarity/maximization, transversality	Concavity for maximization or convexity for minimization; or an HJB verification theorem.
Open-loop Nash differential game	One PMP system per player, with player-specific adjoints and pointwise best responses	Each player's Hamiltonian/objective is concave in its own state-control variables, plus the Nash inequalities hold; for zero-sum games, saddle-point/Isaacs conditions.
Impulse or continuous–impulsive hybrid control	Continuous PMP plus state-jump equation, adjoint-jump equation, impulse-magnitude condition, and sometimes impulse-time condition	Verification inequalities using quasi-variational inequalities (QVIs), or restricted sufficiency for fixed impulse schedules and convex/concave structures.
Numerical forward-backward sweep	Small residual in state, adjoint, and stationarity equations	Not sufficient by itself; should be supported by baseline comparison, sensitivity tests, and unilateral-deviation tests for games.

6.2 Continuous-Time PMP: State, Adjoint, Stationarity, and Transversality

Consider a maximization problem with terminal payoff,

$$\max_{u(\cdot)} \quad J(u) = \Phi(x(T)) + \int_0^T F(x(t), u(t), t)\,\mathrm{d}t, \tag{30}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t), u(t), t), \qquad x(0) = x_0. \tag{31}$$

Define

$$H(x, u, \lambda, t) = F(x, u, t) + \lambda^{\mathsf{T}} f(x, u, t). \tag{32}$$

For an interior normal extremal, PMP gives

$$\dot{x}^*(t) = H_\lambda(x^*, u^*, \lambda^*, t) = f(x^*, u^*, t), \tag{33}$$

$$\dot{\lambda}^*(t) = -H_x(x^*, u^*, \lambda^*, t), \tag{34}$$

$$u^*(t) \in \arg\max_{u \in U} H(x^*(t), u, \lambda^*(t), t), \tag{35}$$

$$\lambda^*(T) = \Phi_x(x^*(T)). \tag{36}$$

If there is no terminal payoff and the terminal state is free, then $\lambda^*(T) = 0$. If the problem is written as a minimization problem, one may either minimize the Hamiltonian $L + \lambda^{\mathsf{T}} f$ or convert the cost to a payoff by changing signs. The mathematics is the same, but the sign convention must be used consistently.

When the admissible set is a box, for example $u_j(t) \in [0, u_{j,\max}]$, an unconstrained stationarity equation $H_{u_j} = 0$ is replaced by a projected control formula. A common pattern is

$$u_j^*(t) = \mathrm{Proj}_{[0,u_{j,\max}]}\left(\widehat{u}_j(t)\right), \tag{37}$$

where $\widehat{u}_j$ is obtained from $H_{u_j} = 0$ before applying bounds. This is exactly the form used in the network examples and the tutorial Python code.

6.3 Sufficiency for Continuous Optimal Control

PMP becomes sufficient under additional structure. For a maximization problem, a standard sufficient condition is:

- the terminal payoff $\Phi(x)$ is concave in x;
- for each t, the Hamiltonian $H(x, u, \lambda^*(t), t)$ is concave in (x, u);
- $u^*(t)$ maximizes the Hamiltonian over the admissible set;
- the state and adjoint equations and transversality condition hold.

Then u^* is a global optimum. For a minimization problem, replace concavity/maximization by convexity/minimization. In many cybersecurity and propagation models, nonlinear infection terms such as $(1 - x_i)\sum_j a_{ij}x_j$ make global concavity difficult to prove. Therefore, PMP is often used to derive a principled candidate strategy, and the quality of the strategy is assessed numerically.

6.4 HJB and the Verification Theorem

PMP gives trajectories and open-loop controls. HJB gives a value function and feedback controls. For a minimization problem

$$V(x,t) = \inf_{u(\cdot)} \left\{ \Phi(x(T)) + \int_t^T L(x(s), u(s), s)\, \mathrm{d}s \right\}, \tag{38}$$

the HJB equation is

$$-V_t(x,t) = \min_{u \in U} \left\{ L(x,u,t) + \nabla_x V(x,t)^{\mathsf{T}} f(x,u,t) \right\}, \qquad V(x,T) = \Phi(x). \tag{39}$$

The feedback policy is

$$u^*(x,t) \in \arg\min_{u \in U} \left\{ L(x,u,t) + \nabla_x V(x,t)^{\mathsf{T}} f(x,u,t) \right\}. \tag{40}$$

The verification theorem says that if a sufficiently smooth function V satisfies the HJB equation and the proposed feedback policy attains the pointwise minimum, then the policy is optimal and V is the optimal value function. This is a sufficient condition, not only a necessary condition.

PMP versus HJB

PMP solves along one trajectory and introduces an adjoint $\lambda(t)$. HJB solves over the state space and introduces a value function $V(x,t)$. Along a smooth optimal trajectory, the two are connected by $\lambda(t) = \nabla_x V(x^*(t),t)$. The advantage of HJB is feedback optimality. The disadvantage is dimensionality: for node-level networks, V depends on all node states.

6.5 HJB/HJI Equations for Games

For a zero-sum attacker-defender game with payoff to the attacker and loss to the defender, the value function may satisfy a Hamilton-Jacobi-Isaacs (HJI) equation:

$$-V_t(x,t) = \max_{a\in A}\min_{d\in D}\left\{F(x,a,d,t) + \nabla_x V(x,t)^{\mathsf{T}} f(x,a,d,t)\right\}. \tag{41}$$

If the max-min and min-max values coincide, the Isaacs condition holds and a saddle-point feedback equilibrium can be characterized.

For a nonzero-sum feedback Nash game, there is generally one value function per player. A formal system is

$$-\frac{\partial V_i}{\partial t}(x,t) = F_i(x,u_1^*,\dots,u_m^*,t) + \nabla_x V_i(x,t)^{\mathsf{T}} f(x,u_1^*,\dots,u_m^*,t), \qquad i=1,\dots,m, \tag{42}$$

where each u_i^* is a best response in the ith Hamiltonian. These coupled PDEs are difficult even for low-dimensional systems, and usually infeasible for full node-level network propagation models.

6.6 Impulse HJB and Quasi-Variational Inequalities

Feedback impulse control has a dynamic-programming form. For a minimization problem, define the intervention operator

$$(\mathcal{M}V)(x,t) = \inf_{v\in V}\left\{K(x,v,t) + V\big(x+g(x,v,t),t\big)\right\}, \tag{43}$$

where K is the impulse cost and $x+g(x,v,t)$ is the post-impulse state. The value function satisfies a quasi-variational inequality (QVI) of the form

$$\min\left\{-V_t(x,t) - \inf_{u\in U}\left[L(x,u,t) + \nabla_x V(x,t)^{\mathsf{T}} f(x,u,t)\right],\ V(x,t) - (\mathcal{M}V)(x,t)\right\} = 0. \tag{44}$$

The first term is the continuation condition: do not impulse, continue with continuous control. The second term compares the value of waiting with the value after immediate intervention. The impulse region is where $V(x,t) = (\mathcal{M}V)(x,t)$; the continuation region is where the HJB part holds. Continuous–impulsive feedback games lead to HJI-QVI systems with one or more value functions. These equations provide strong verification principles, but they are rarely solved directly for high-dimensional node-level networks.

6.7 PMP with Impulsive State Jumps

Now consider an impulse or continuous–impulsive hybrid model with fixed impulse times $t_1, \dots, t_N$:

$$\dot{x}(t) = f(x(t), u(t), t), \qquad t \notin \{t_1, \dots, t_N\}, \tag{45}$$

$$x(t_l^+) = x(t_l^-) + g(x(t_l^-), v_l, t_l), \qquad l = 1, \dots, N. \tag{46}$$

The payoff is

$$J = \Phi(x(T)) + \int_0^T F(x(t), u(t), t)\,\mathrm{d}t + \sum_{l=1}^{N} G(x(t_l^-), v_l, t_l). \tag{47}$$

Define the continuous Hamiltonian and the impulse Hamiltonian by

$$H(x, u, \lambda, t) = F(x, u, t) + \lambda^{\mathsf{T}} f(x, u, t), \tag{48}$$

$$H^I(x, v, \lambda^+, t_l) = G(x, v, t_l) + (\lambda^+)^{\mathsf{T}} g(x, v, t_l), \tag{49}$$

where $\lambda^+ = \lambda(t_l^+)$ is the adjoint just after the jump. The PMP contains the usual continuous equations between impulse times and the following jump and impulse conditions:

$$x(t_l^+) = x(t_l^-) + g(x(t_l^-), v_l, t_l), \tag{50}$$

$$\lambda(t_l^-) = \lambda(t_l^+) + \frac{\partial H^I}{\partial x}(x(t_l^-), v_l, \lambda(t_l^+), t_l), \tag{51}$$

$$v_l^* \in \arg\max_{v \in V} H^I(x(t_l^-), v, \lambda(t_l^+), t_l). \tag{52}$$

For a minimization problem, the corresponding impulse condition uses an argmin under the minimization sign convention. The adjoint jump condition is the theoretical correction that accounts for the fact that the state does not evolve continuously at t_l.

Cleanup impulse in a node-level network

Suppose an emergency cleanup at time t_l reduces selected node states by a fraction $z_i \in [0, 1]$:

$$x_i(t_l^+) = (1 - z_i)x_i(t_l^-), \qquad g_i(x, z) = -z_i x_i. \tag{53}$$

In a minimization problem with impulse cost $K(z) = c_0 + \frac{1}{2}c_z \sum_i z_i^2$, the impulse Hamiltonian is

$$H^I(x, z, \lambda^+, t_l) = K(z) + (\lambda^+)^T g(x, z). \tag{54}$$

The first-order condition balances marginal cleanup benefit, represented by $-\lambda_i^+ x_i$, against marginal impulse cost $c_z z_i$, followed by projection to $[0, 1]$. This is the jump analogue of the projected continuous-control formula.

If the impulse times are also decision variables and the model is autonomous or has no explicit time-dependence in the impulse reward, a common first-order condition is continuity of the continuous Hamiltonian across an optimal impulse time:

$$H(x(t_l^-), u(t_l^-), \lambda(t_l^-), t_l) = H(x(t_l^+), u(t_l^+), \lambda(t_l^+), t_l). \tag{55}$$

With explicit time-dependence, additional terms involving $\partial H^I / \partial t_l$ may appear. In practice, solving simultaneously for impulse magnitudes, impulse times, and the number of impulses is

much harder than solving a continuous PMP system.

6.8 Continuous–Impulsive Hybrid PMP and Nested Optimization

Continuous–impulsive hybrid control combines two types of first-order conditions:

1. **Flow conditions**: state equation, adjoint equation, and continuous-control stationarity between impulses.
2. **Jump conditions**: state jump, adjoint jump, impulse-magnitude stationarity, and possibly impulse-time stationarity.
3. **Discrete-event conditions**: selection of impulse number N and target set, which is usually not handled by classical smooth PMP alone.

For a fixed N and fixed impulse schedule, the continuous–impulsive PMP is close to a continuous PMP system with extra jump equations. When N, impulse times, and target nodes are also optimized, the problem becomes a nested continuous-discrete optimization problem:

$$\max_{N} \max_{0<t_1<\cdots<t_N<T} \max_{u(\cdot),v_1,\ldots,v_N} J(u, v_1, \ldots, v_N, t_1, \ldots, t_N, N). \tag{56}$$

This explains why many network propagation studies use restricted impulse schedules, candidate-time search, greedy target selection, or learning-based parameterized action spaces. These approaches do not replace the PMP; they make the PMP-inspired problem computationally tractable.

6.9 Impulse and Continuous–Impulsive Hybrid Differential Games

In a continuous–impulsive hybrid differential game, each player has its own continuous Hamiltonian and, if that player can trigger jumps, its own impulse Hamiltonian. For a two-player nonzero-sum game, player i has

$$H_i(x, u_1, u_2, \lambda_i, t) = F_i(x, u_1, u_2, t) + \lambda_i^{\mathsf{T}} f(x, u_1, u_2, t), \tag{57}$$

$$H_i^I(x, v_i, \lambda_i^+, t_l) = G_i(x, v_i, t_l) + (\lambda_i^+)^{\mathsf{T}} g_i(x, v_i, t_l). \tag{58}$$

The equilibrium conditions combine mutual best responses:

$$u_i^*(t) \in \arg\max_{u_i} H_i(x^*, u_i, u_{-i}^*, \lambda_i, t), \tag{59}$$

$$v_{i,l}^* \in \arg\max_{v_i} H_i^I(x^*(t_l^-), v_i, \lambda_i(t_l^+), t_l), \tag{60}$$

with state and adjoint flow/jump equations. In nonzero-sum nonlinear network games, these conditions are normally necessary candidate-equilibrium conditions. Full sufficiency is much harder and usually requires restrictive concavity and equilibrium verification assumptions.

Practical interpretation for network defense

A continuous control such as patching modifies the derivative: it slowly changes the rate at which compromise is removed. An impulse such as emergency cleanup modifies the state: it immediately reduces selected node compromise probabilities. The adjoint jump says that an impulse also changes the future shadow value of the state. This is why impulse timing and target selection can be strategically important even when the visible state reduction looks similar.

6.10 A Short Theoretical Checklist

When presenting an optimal control or differential game model, the theoretical part is clearer if it includes the following items:

1. State clearly whether the problem is maximization or minimization, and keep the Hamiltonian sign convention consistent.
2. Specify whether controls are open-loop, feedback, sampled-data, impulse, or continuous–impulsive hybrid.
3. Write the Hamiltonian(s). For games, write one Hamiltonian per player.
4. Derive state, adjoint, terminal/transversality, and stationarity conditions.
5. For impulse or continuous–impulsive hybrid models, include state jump, adjoint jump, impulse Hamiltonian, and event-time condition if times are optimized.
6. State whether the conditions are necessary only, or whether extra convexity/concavity/HJB verification arguments make them sufficient.
7. In numerical work, report baselines, sensitivity, and convergence or residual checks.

7 Degree-Level and Node-Level Examples

This section gives concrete model templates. They are not the only possible choices; they are clean starting points for research students to extend toward APT, malware, rumor, propaganda, epidemic, or cyberwar models.

7.1 Degree-Level Optimal Control: Degree-k Controlled SIS Spreading

For degree class $k \in \mathcal{K}$, let $x_k(t)$ be the infected or compromised fraction among nodes whose network degree is exactly k. The degree distribution $P(k)$ and average degree $\langle k \rangle$ are computed from the full input network before the control problem is solved. A defender chooses one control $u_k(t)$ per degree class:

$$\dot{x}_k = \beta k(1 - x_k)\Theta(t) - (\delta + u_k)x_k, \qquad \Theta(t) = \frac{\sum_{\ell \in \mathcal{K}} \ell P(\ell) x_\ell(t)}{\langle k \rangle}. \tag{61}$$

The population-weighted defender objective is

$$\min_{u(\cdot)} \int_0^T \left[q \sum_{k \in \mathcal{K}} P(k) x_k(t) + \frac{r}{2} \sum_{k \in \mathcal{K}} P(k) u_k(t)^2 \right] \mathrm{d}t. \tag{62}$$

The weights $P(k)$ matter: controlling a rare degree class should not be counted the same as controlling a class containing most of the nodes. With the Hamiltonian convention for minimization, the projected PMP update used in the code is

$$u_k^*(t) = \mathrm{Proj}_{[0,u_{\max}]} \left(\frac{\lambda_k(t) x_k(t)}{r P(k)} \right), \qquad k \in \mathcal{K}. \tag{63}$$

This is the main adjustment from a compartment-level toy model: arrays are organized by observed degree k, and the objective is weighted by the empirical degree distribution.

7.2 Node-Level Optimal Control: Targeted Patching

Let $x_i(t)$ be the compromise probability of node i. The network adjacency matrix is A. A defender chooses node-specific patching $u_i(t)$:

$$\dot{x}_i = \beta(1 - x_i) \sum_{j=1}^{n} a_{ij} x_j - (\delta + u_i) x_i. \tag{64}$$

The defender minimizes

$$\int_0^T \left[q \sum_i x_i(t) + \frac{1}{2} r \sum_i u_i(t)^2 \right] \mathrm{d}t. \tag{65}$$

The Hamiltonian minimization yields a projected update

$$u_i^*(t) = \mathrm{Proj}_{[0,u_{\max}]} \left(\frac{\lambda_i(t) x_i(t)}{r} \right). \tag{66}$$

In practice, high infection pressure and high adjoint value tend to produce larger u_i for a node. This is how PMP naturally generates targeted resource allocation.

7.3 Degree-Level Attacker-Defender Differential Game

The same degree-indexed structure gives a nonzero-sum differential game. The attacker chooses $a_k(t)$ to increase spreading in degree class k, while the defender chooses $d_k(t)$ to increase removal or repair:

$$\dot{x}_k = \beta(1 + a_k) k (1 - x_k) \Theta(t) - (\delta + d_k) x_k. \tag{67}$$

A population-weighted open-loop Nash formulation is

$$J_A(a, d) = \int_0^T \left[\rho_A \sum_k P(k) x_k - \frac{c_A}{2} \sum_k P(k) a_k^2 \right] \mathrm{d}t, \tag{68}$$

$$J_D(a, d) = \int_0^T \left[-\rho_D \sum_k P(k) x_k - \frac{c_D}{2} \sum_k P(k) d_k^2 \right] \mathrm{d}t. \tag{69}$$

The code solves the PMP necessary conditions with one adjoint vector for the attacker and one adjoint vector for the defender. The Hamiltonian stationarity conditions produce projected

degree-class updates such as

$$a_k^* = \mathrm{Proj}_{[0,a_{\max}]}\left(\frac{\lambda_k^A \beta k(1-x_k)\Theta}{c_A P(k)}\right), \qquad d_k^* = \mathrm{Proj}_{[0,d_{\max}]}\left(-\frac{\lambda_k^D x_k}{c_D P(k)}\right). \tag{70}$$

7.4 Node-Level Attacker-Defender Differential Game

At node level, let $v_i(t)$ be attacker effort around node i and $u_i(t)$ be defender effort. A clean template is

$$\dot{x}_i = \beta(1+v_i)(1-x_i)\sum_{j=1}^{n} a_{ij}x_j - (\delta+u_i)x_i. \tag{71}$$

The payoffs are

$$J_A(v,u) = \int_0^T \left[p\sum_i x_i(t) - \frac{1}{2}c_v\sum_i v_i(t)^2\right]\,\mathrm{d}t, \tag{72}$$

$$J_D(v,u) = \int_0^T \left[-q\sum_i x_i(t) - \frac{1}{2}c_u\sum_i u_i(t)^2\right]\,\mathrm{d}t. \tag{73}$$

This model is a direct node-level extension of the degree-level attacker-defender game. It is suitable for small to medium networks or for simulation experiments. For large networks, dimension reduction, graph neural networks, mean-field approximations, or learning-based policies may be needed.

7.5 Continuous–Impulsive Node-Level Control: Continuous Patching plus Emergency Cleanup

Between impulses, use node-level controlled dynamics. At selected impulse times, clean selected nodes instantaneously:

$$\dot{x}_i = \beta(1-x_i)\sum_{j=1}^{n} a_{ij}x_j - (\delta+u_i)x_i, \qquad t \notin \{t_l\}, \tag{74}$$

$$x_i(t_l^+) = (1-z_{i,l})x_i(t_l^-), \qquad \text{for nodes receiving an impulse.} \tag{75}$$

A practical objective is

$$\min \int_0^T \left[q\sum_i x_i(t) + \frac{1}{2}r\sum_i u_i(t)^2\right]\,\mathrm{d}t + \sum_l \left(c_0 + \frac{1}{2}c_z\|z_l\|^2\right). \tag{76}$$

This model captures routine defense plus occasional high-cost intervention. The tutorial code uses a transparent simulation with fixed impulse times applied to high-degree nodes. This is not a full impulse PMP solver; it illustrates how continuous ODE segments and jump maps are combined in a continuous–impulsive computation.

8 Numerical Solution Methods and Practical Cautions

8.1 Forward-Backward Sweep for Games

For a two-player open-loop Nash game, forward-backward sweep can be structured as follows.

```
initialize u(t), v(t)
repeat:
    # common state trajectory
    x(t) = forward solve of dx/dt = f(x, u, v)

    # player-specific adjoint systems
    lambda_1(t) = backward solve of dlambda_1/dt = -dH_1/dx
    lambda_2(t) = backward solve of dlambda_2/dt = -dH_2/dx

    # pointwise best responses
    u_new(t) = argmax_u H_1(x, u, v, lambda_1)
    v_new(t) = argmax_v H_2(x, u, v, lambda_2)

    # damping / relaxation
    u = theta * u_new + (1 - theta) * u
    v = theta * v_new + (1 - theta) * v
until convergence or maximum iterations
```

Listing 3: Forward-backward sweep for an open-loop Nash game.

8.2 Direct Optimization and Collocation

Instead of deriving adjoint equations, one can discretize the state and control and solve a nonlinear programming problem. This is often easier to implement for complex models, bounds, and continuous–impulsive constraints. However, it may scale poorly and can still converge to local optima. For impulse times, direct search or mixed-integer formulations may be used for small problems.

8.3 Why HJB is Difficult in Network Models

Feedback optimal control and feedback differential games are theoretically attractive because the strategy can adapt to the observed state. The value function $V(x, t)$, however, is defined over the whole state space. For node-level network models with n nodes, x is n-dimensional. A grid-based HJB method suffers from the curse of dimensionality: even 20 nodes can be too large for classical gridding. This motivates approximate dynamic programming, reduced-order modeling, and other scalable approximation methods.

8.4 Model Checking and Baseline Comparison

- Always compare a computed control/game strategy against no control, constant control, random control, greedy control, and simple degree/centrality-based allocation.
- Check sensitivity to β, δ, cost weights, horizon T, and initial infection.
- For games, test unilateral deviations to see whether the computed pair behaves like a Nash equilibrium numerically.
- For continuous–impulsive policies, separate the value of continuous effort from the value of impulses by running ablation experiments.
- Report convergence curves or final residuals for forward-backward sweep when possible.

9 Python Tutorial Code: Simple First, Then Research-Code-Style Examples

The tutorial code is organized for teaching and reproducibility. It avoids a large package structure, but still separates the learning path into two files:

$$\text{minimal degree-}k\text{ example} \quad \Longrightarrow \quad \text{compact research-code-style tutorial script.}$$

The first file is short enough to read in one sitting. The second file keeps all examples in one place, so the reader can see how graph loading, degree distribution, ODE integration, PMP updates, game updates, impulse simulation, plotting, and command-line options fit together.

9.1 Tutorial Files

In this public repository, the commands in this section assume that you first move into `examples/foundations/`. From the repository root, you can also run all tutorial examples with `pythonrun_all.py--skip-reference`.

Table 6: Code and guide files in the tutorial package.

File	Purpose
`code/simple_degree_k_control.py`	Minimal degree-k optimal-control script. It loads a graph, computes N_k, $P(k)$, and $\langle k\rangle$, solves one degree-k PMP problem, and saves diagnostic figures.
`code/network_control_examples.py`	Main tutorial script containing degree-k optimal control, degree-k attacker-defender game, node-level optimal control, node-level game, continuous-impulse simulation, graph loading, adjacency conversion, and plotting.
`reference_repository_guide.md`	Explains how the three reference research-code repositories map to the mathematical workflow in this note: graph preprocessing, state dynamics, adjoints, Hamiltonian updates, impulse policies, payoffs, and baselines.
`../reference/download_reference_repositories.sh`	Optional helper script that downloads the three reference GitHub repositories into `../reference/reference_repositories_upstream/`.

9.2 Run the Simple Degree-k Example

The simple script demonstrates the central point of degree-level modeling: the state and control arrays are indexed by observed degree class, not by compartment labels. If the observed degrees are $k_1, \ldots, k_q$, then

$$X[:, j] = x_{k_j}(t), \qquad U[:, j] = u_{k_j}(t), \qquad j = 1, \ldots, q.$$

The infection pressure is computed from the empirical distribution:

$$\Theta(t) = \frac{\sum_j k_j P(k_j) x_{k_j}(t)}{\langle k\rangle}.$$

```
pip install -r requirements.txt

python code/simple_degree_k_control.py

python code/simple_degree_k_control.py \
  --edge-list sample_data/sample_edges.csv \
  --delimiter , --has-header \
  --source-col source --target-col target

python code/simple_degree_k_control.py \
  --adjacency-csv sample_data/sample_adjacency.csv
```

Listing 4: Run the minimal degree-k optimal-control script.

The core numerical loop is a forward-backward sweep. Given a current control $u_k(t)$, it solves the state equation forward, the adjoint equation backward, and then applies a projected Hamiltonian update:

```
# k       = observed degree values, e.g. [2, 3, 4, 7]
# p       = empirical P(k)
# X, L    = state and adjoint arrays with columns indexed by degree class k
p_safe = np.maximum(p, 1e-12)
candidate_U = L * X / (control_weight * p_safe)
U = damping * clip(candidate_U, 0.0, u_max) + (1.0 - damping) * U_old
```

Listing 5: Degree-level projected PMP control update.

9.3 Run the Main Examples

The main tutorial script extends the same workflow to games, node-level models, and continuous-impulse simulations. The command-line interface makes it possible to run all examples or only one group.

```
python code/network_control_examples.py
python code/network_control_examples.py --examples degree
python code/network_control_examples.py --examples node
python code/network_control_examples.py --examples continuous-impulse
```

Listing 6: Run the main tutorial examples.

For realistic network data, the script accepts edge lists, adjacency matrices, and common graph formats:

```
python code/network_control_examples.py \
  --edge-list data/edges.csv \
  --delimiter , --has-header \
  --source-col source --target-col target --weight-col weight \
  --max-nodes 50

python code/network_control_examples.py --graph-file data/network.graphml
python code/network_control_examples.py --graph-file data/network.gexf
python code/network_control_examples.py --graph-file data/network.gml
python code/network_control_examples.py --graph-file data/network.net

python code/network_control_examples.py --adjacency-csv data/A.csv
```

Listing 7: Use realistic network data.

9.4 How the Reference Repositories Fit In

The three reference repositories correspond to the author's co-authored cyber/network-control publications: two papers in IEEE TIFS and one paper in IEEE TCSS. They follow a common research-code pattern: network preprocessing, forward dynamics, backward adjoints, control or strategy update, payoff evaluation, and comparison/baseline experiments. The tutorial code uses the same pattern with smaller equations and shorter functions.

Table 7: Mapping from the reference repositories to the tutorial examples.

Repository	**Representative files/functions**	**Interpretation in this note**
OpinionMalware (IEEE TIFS 2025)	Repository `OpinionMalware_TIFS_2025_Code`; `network.py`, `opinionMalware.py`; network construction, normalization, coupled forward malware/opinion dynamics, backward adjoints, impulse strategy search, payoff.	Node-level coupled dynamics with impulses. The continuous-impulse simulation in the tutorial code is a minimal bridge before reading the full malware-opinion model.
PropagandaWar (IEEE TIFS 2024)	Repository `PropagandaWar_TIFS_2024_Code`; `demo_network.py`, `propWar.py`, `comparison.py`; graph loading, degree distribution, forward/backward Nash sweep, continuous-plus-impulse strategy update, random-strategy comparison.	Closest to the degree-k differential-game example. The same blocks appear: $P(k)$ preprocessing, state equations, adjoints, bounded strategy updates, and payoff comparison.
Propaganda (IEEE TCSS 2025)	Repository `Propaganda_TCSS_2025_Code`; `prop_network.py`, `prop_propaganda.py`, `prop_cntrlComparison.py`, `prop_noAwareness.py`; network loading, impulse policy search, profit/payoff, random and no-awareness baselines.	Impulse-control workflow and ablation studies. The tutorial function `simulate_continuous_impulse` gives the basic state-jump structure before the full paper-specific implementation.

The reference code can be downloaded with:

```
bash ../reference/download_reference_repositories.sh
```

Listing 8: Download the reference repositories.

The file `reference_repository_guide.md` gives a reading map from the reference repositories to the mathematical ingredients of PMP, impulse control, and continuous–impulsive hybrid control.

9.5 Representative Outputs

The figures below are curated repository assets generated from the tutorial scripts. Fresh runs write detailed CSV files and additional figures under `artifacts/`.

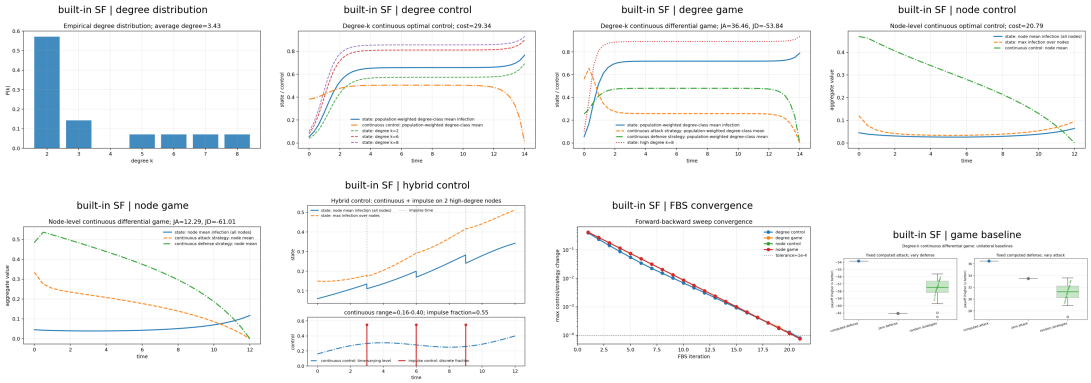


Figure 1: Visual index of the foundation examples, including degree-level control/game, node-level variants, convergence diagnostics, and continuous-impulse behavior.

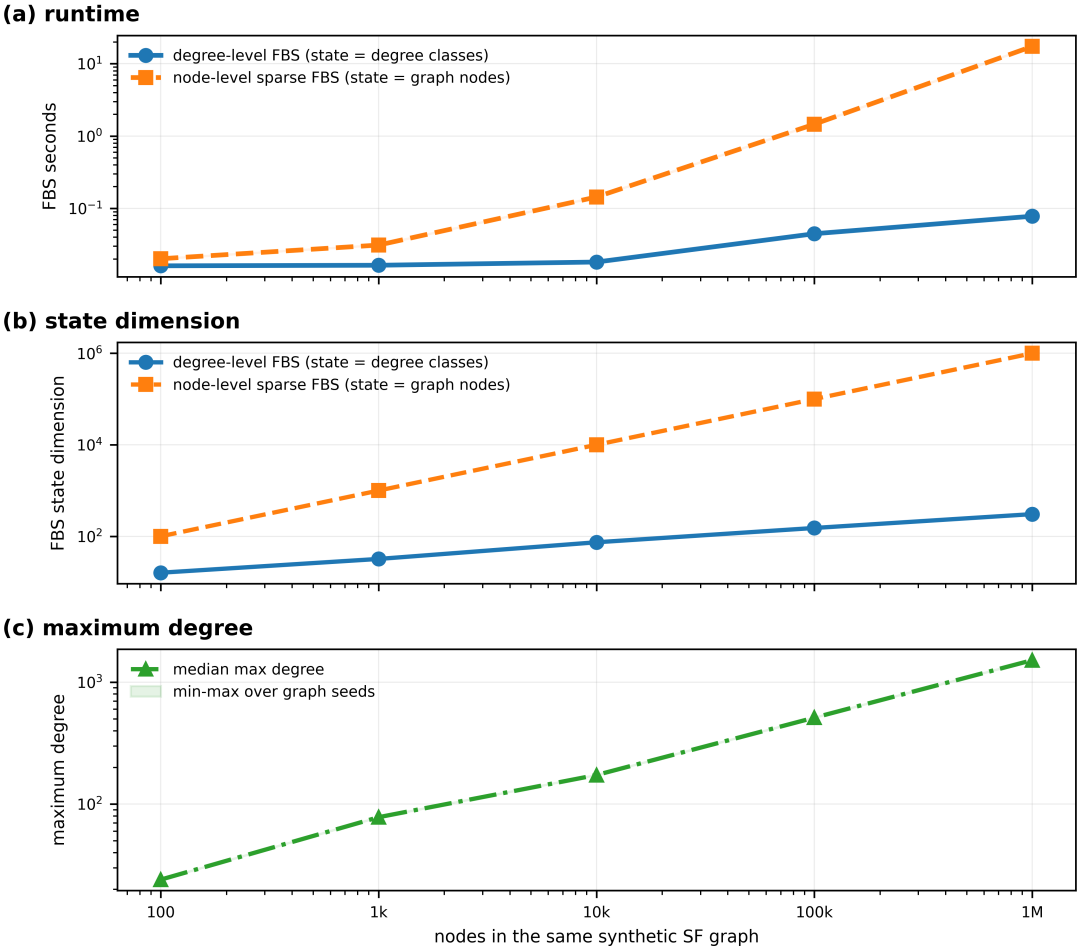


Figure 2: Degree-level versus sparse node-level FBS timing on matched synthetic scale-free networks.

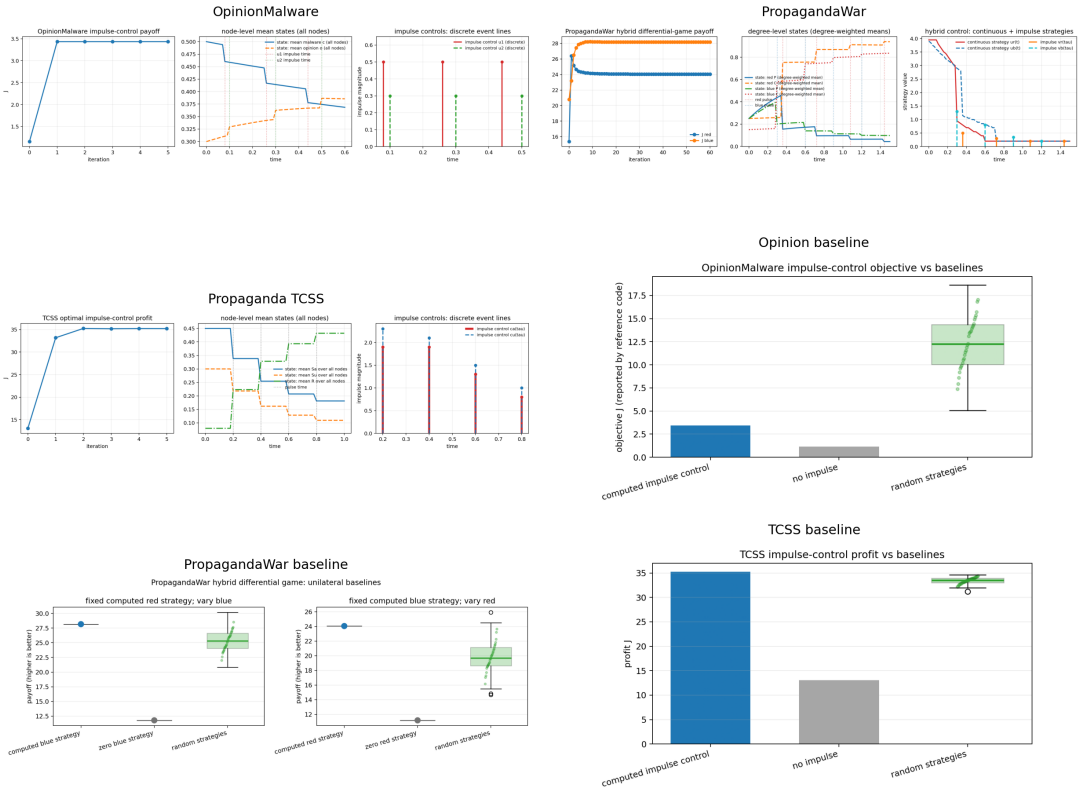


Figure 3: Reference-repository smoke-run contact sheet for the paper-level code snapshots.

10 Glossary and References

10.1 Glossary

Term	Meaning
State $x(t)$	Variables describing the system at time t, such as node infection probabilities.
Control $u(t)$	Decision variable chosen by a controller over time.
Payoff/cost functional J	Cumulative objective over the time horizon.
Hamiltonian H	Immediate payoff plus adjoint-weighted state dynamics.
Adjoint $\lambda(t)$	Shadow value of the state; solved backward in time.
PMP	Pontryagin Maximum Principle; necessary conditions for optimal control or open-loop games.
HJB	Hamilton-Jacobi-Bellman equation; dynamic programming equation for feedback optimal control.
Nash equilibrium	Strategy profile where no player benefits from unilateral deviation.
Stackelberg equilibrium	Leader-follower equilibrium with sequential decision structure.
Impulse control	Discrete intervention causing instantaneous state jumps.
Continuous–impulsive hybrid control	Flow controls plus explicit impulse resets.
Degree-level model	Model that groups nodes by degree/type rather than tracking each node.
Node-level model	Model that tracks each node and uses the adjacency matrix.

10.2 References and Source Notes

[1] Isaacs, R. (1965). *Differential Games*. John Wiley and Sons.

[2] Basar, T., and Zaccour, G. (eds.) (2018). *Handbook of Dynamic Game Theory*. Springer.

[3] Simaan, M., and Cruz, J. B. (1973). Sampled-data Nash controls in non-zero-sum differential games. *International Journal of Control*, 17(6), 1201-1209.

[4] Sadana, U., Reddy, P. V., and Zaccour, G. (2021). Nash equilibria in nonzero-sum differential games with impulse control. *European Journal of Operational Research*, 295, 792-805.

[5] Yang, L.-X., Li, P., Yang, X., Xiang, Y., and Zhou, W. (2018). A differential game approach to patch injection. *IEEE Access*, 6, 58924-58938.

[6] Yang, L.-X., Li, P., Zhang, Y., Yang, X., Xiang, Y., and Zhou, W. (2019). Effective repair strategy against advanced persistent threat: a differential game approach. *IEEE Transactions on Information Forensics and Security*, 14(7), 1713-1728.

[7] Yang, L.-X., Huang, K., Yang, X., Zhang, Y., Xiang, Y., and Zhou, Y. Y. (2021). Defense against advanced persistent threat through data backup and recovery. *IEEE Transactions on Network Science and Engineering*, 8(3), 2001-2013.

[8] Yang, L.-X., Li, P., Yang, X., and Tang, Y. Y. (2020). Simultaneous benefit maximization of conflicting opinions: modeling and analysis. *IEEE Systems Journal*, 14(2), 1623-1634.

[9] Huang, D. W., Yang, L.-X., Li, P., Yang, X., and Tang, Y. Y. (2021). Developing cost-effective rumor-refuting strategy through game-theoretic approach. *IEEE Systems Journal*, 15(4), 5034-5045.

[10] Cheng, X., Yang, L.-X., Li, G., Ma, Z., Zhu, T., Wang, L., and Duan, S. (2025). Modeling and Mitigating Social Engineering Malware: Integrating Malware-Opinion Dynamics With Optimal Impulse Control Approaches. Code repository: https://github.com/XiaojuanCheng/OpinionMalware_TIFS_2025_Code.

[11] Cheng, X., Yang, L.-X., Zhu, Q., Gan, C., Yang, X., and Li, G. (2024). Cost-Effective Hybrid Control Strategies for Dynamical Propaganda War Game. Code repository: https://github.com/XiaojuanCheng/PropagandaWar_TIFS_2024_Code.

[12] Cheng, X., Yang, L.-X., Zhu, Q., Gan, C., and Li, G. (2025). Impulse Strategies for Suppressing Cyber Propaganda With Awareness. Code repository: https://github.com/XiaojuanCheng/Propaganda_TCSS_2025_Code.

Source note

This English tutorial note was consolidated from two uploaded source tutorial notes: a differential-game tutorial note and an optimal-hybrid-control tutorial note. The source materials cover the origins and classification of differential games, open-loop Nash differential games, Pontryagin-type optimality systems, forward-backward sweep, single-impulse games, continuous–impulsive hybrid control with state jumps, impulse Hamiltonians, and the nested optimization structure for impulse number, impulse times, and impulse magnitudes. The additional implementation sections translate these models into degree-level and node-level network workflows, including edge-list-to-adjacency conversion for realistic datasets, NetworkX/igraph graph support, ODE helper functions, and degree-distribution preprocessing for degree-k models.

A Tutorial Code Files

The package contains the following runnable code and guide files:

- `code/simple_degree_k_control.py`: minimal degree-k optimal-control example.
- `code/network_control_examples.py`: main tutorial script for degree-level games, node-level models, and continuous-impulse simulation.
- `README.md`: installation, commands, and usage notes.
- `reference_repository_guide.md`: mapping between the reference GitHub repositories and the mathematical workflow in this guide.

- `../reference/download_reference_repositories.sh`: optional script to download the reference repositories.

```
pip install -r requirements.txt

python code/simple_degree_k_control.py
python code/simple_degree_k_control.py \
  --edge-list sample_data/sample_edges.csv \
  --delimiter , --has-header --source-col source --target-col target

python code/network_control_examples.py
python code/network_control_examples.py --examples degree
python code/network_control_examples.py --adjacency-csv sample_data/sample_adjacency.csv

bash ../reference/download_reference_repositories.sh
```

Listing 9: Core commands for the tutorial code.

This two-file code structure keeps the tutorial readable: the first script shows the degree-k PMP logic, and the main tutorial script shows how the same logic extends to games, node-level adjacency models, and continuous-impulsive interventions.